

La stack pour passer de l'idée à l'app en ligne : Claude, Supabase, Vercel, Stripe, GitHub Actions

CLAUDE · 7 MIN DE LECTURE

Sommaire

- Introduction
- La stack complète pour construire et mettre en ligne un vrai produit
- Le vrai problème, celui dont on ne parle pas assez
- Les 5 briques : rôle, intérêt, coût
- L'ordre à suivre pour vous lancer dès ce soir
- Le prompt pour démarrer (à copier dans Claude)
- Quand s'en servir, et ce qu'il faut savoir avant
- Le fichier complet
- Pour aller plus loin

Introduction

Cinq briques technologiques et un agent de code pour passer de l'idée à un produit en ligne capable d'encaisser de vrais paiements : à quoi sert chaque brique, laquelle mérite vraiment votre attention, laquelle vous pouvez ignorer pour l'instant, l'ordre à suivre pour vous lancer ce soir, et le prompt Claude prêt à l'emploi pour démarrer.

01. La stack complète pour construire et mettre en ligne un vrai produit

Claude pour écrire le code. Supabase pour le back-end. Vercel pour le déploiement. Stripe pour les paiements. GitHub Actions pour l'intégration et le déploiement continu. C'est tout ce dont vous avez besoin pour transformer une idée en produit en ligne capable d'encaisser de véritables paiements. Après des années à créer des applications, ce qui prenait autrefois des mois se règle aujourd'hui en quelques jours. Voici chaque brique en détail, son utilité, ce qui mérite vraiment le détour et ce que vous pouvez laisser de côté au départ, avec l'ordre exact à suivre pour vous lancer dès ce soir.

02. Le vrai problème, celui dont on ne parle pas assez

Le vrai frein n'est presque jamais technique. C'est le nombre de décisions à prendre avant même de commencer à coder : quel framework choisir, quelle base de données, quel hébergeur, comment encaisser les paiements, comment déployer sans tout casser. Chaque choix agit comme une porte fermée, et beaucoup de gens restent bloqués devant. Cette stack règle justement ce problème : cinq briques conçues pour fonctionner ensemble, chacune avec un palier gratuit pour démarrer, et un agent de code qui les assemble à votre place. Vous arrêtez de multiplier les choix, et vous passez directement à la construction.

03. Les 5 briques : rôle, intérêt, coût

1. Claude : le code. C'est l'élément qui change véritablement la donne. Avec Claude Code, vous décrivez ce que vous voulez obtenir, et il rédige, corrige et explique le code, fichier après fichier. Vous ne partez jamais d'une page blanche : Claude devient votre développeur, vous gardez le rôle de réalisateur du projet.

2. Supabase : le back-end. Base de données (Postgres), authentification et stockage de fichiers opérationnels en quelques minutes, avec un palier gratuit généreux pour se lancer. C'est ici que résident vos utilisateurs et leurs données.

3. Vercel : le déploiement. Votre application mise en ligne en une seule commande (ou un simple push git), avec une URL de production disponible sur un palier gratuit au démarrage. Vous pouvez déployer aussi souvent que vous le souhaitez, sans vous soucier de la gestion des serveurs.

4. Stripe : les paiements. Pour encaisser réellement de l'argent : abonnements ou paiements ponctuels, avec des pages de paiement déjà prêtes. Pas d'abonnement mensuel fixe, juste une commission par transaction (dont le détail varie selon votre pays).

5. GitHub Actions : l'intégration et le déploiement continus. Votre filet de sécurité : à chaque modification, vos tests s'exécutent automatiquement et le déploiement se déclenche sans intervention manuelle. Vous arrêtez de risquer de casser la production à la main. Palier gratuit disponible pour les petits projets.

Ce qui mérite vraiment votre attention dès le départ : Claude (le véritable levier) et Supabase (qui vous fait gagner des semaines entières). Ce que vous pouvez ignorer au début : GitHub Actions tant que vous travaillez seul sur un petit projet (le déploiement manuel via Vercel suffit largement), et Stripe tant que vous n'avez encore rien à vendre. Ajoutez ces briques une fois que le produit existe réellement, pas avant.

04. L'ordre à suivre pour vous lancer dès ce soir

1. **Claude** pose la structure du projet : vous décrivez l'app, il construit l'architecture et le premier écran.
2. **Supabase** : vous connectez la base de données et l'authentification, Claude écrit la connexion correspondante.
3. **Vercel** : vous mettez en ligne une première version, même minimale. Voir son app fonctionner réellement change tout.
4. **Stripe** : dès qu'il y a quelque chose à vendre, vous ajoutez le système de paiement.
5. **GitHub Actions** : quand vous voulez automatiser complètement tests et déploiement plutôt que de le faire à la main.

05. Le prompt pour démarrer

À copier directement dans Claude.

Collez ce prompt dans Claude. Il ne construit pas tout en une seule fois : il commence par vous poser les bonnes questions, puis avance brique par brique, en validant chaque étape avec vous.

Prompt à copier (Lancez votre stack avec Claude) :

PROMPT À COPIER

Tu es mon développeur. On va construire et mettre en ligne une petite app avec cette stack : Claude (toi, le code), Supabase (base + auth), Vercel (déploiement), Stripe (paiements), GitHub Actions (CI/CD). Je ne code pas moi-même, je dirige.

Ne construis pas tout d'un coup. D'abord, pose-moi ces questions, une par une :

1. Que fait l'app, en une phrase, et pour qui ?

2. Le parcours principal, du premier écran au résultat ?
3. Quelles données faut-il stocker (les "tables" Supabase et leurs champs) ?
4. Faut-il des comptes utilisateurs ? Un paiement (abonnement ou achat unique) ?
5. Sur quel système je travaille, et qu'est-ce qui est déjà installé ?

Ensuite, avance BRIQUE PAR BRIQUE, en me demandant confirmation à chaque étape :

- Étape 1 : structure du projet + premier écran. On teste avant d'avancer.
- Étape 2 : Supabase (schéma des tables + auth). Montre-moi le SQL avant de l'appliquer.
- Étape 3 : déploiement Vercel (la commande exacte, une seule).
- Étape 4 : Stripe (paiement), seulement si j'ai dit oui. Jamais de clé secrète en clair : réfère-toi aux variables par leur nom, mets-les dans les variables d'environnement.
- Étape 5 : GitHub Actions (tests + déploiement auto), seulement quand je le demande.

Règles : une commande à la fois, tu expliques ce qu'elle fait ; tu ne me fais jamais coller un secret dans le chat ; à la moindre erreur, tu lis le vrai message et tu diagnostiques au lieu de deviner. Commence par les questions.

06. Quand s'en servir, et ce qu'il faut savoir avant

Cette stack démarre pour quelques euros par mois grâce aux paliers gratuits, et Stripe ne prélève sa commission qu'au moment où vous vendez réellement quelque chose. Le vrai coût n'est donc pas l'outil en lui-même, mais votre temps et votre capacité à savoir précisément quoi construire. L'agent écrit le code, mais la responsabilité de ce que vous mettez en ligne reste la vôtre : relisez attentivement tout ce qui touche aux paiements et aux données de vos utilisateurs, et ne laissez jamais traîner une clé secrète dans un fichier ou dans un prompt. Pour une vraie mise en production qui encaisse de l'argent, faites relire les parties sensibles par quelqu'un qui sait coder. « Ça fonctionne à l'écran » ne veut pas encore dire « c'est sécurisé et solide ».

08. Pour aller plus loin

Pour construire votre app écran par écran, sans écrire une ligne de code vous-même, direction le guide Vibe coding 101.

À retenir

Cinq briques, un agent, et vous arrêtez de multiplier les choix pour enfin construire.

Claude écrit le code, Supabase stocke les données, Vercel déploie, Stripe encaisse, GitHub Actions surveille. Commencez par les deux qui comptent vraiment (Claude + Supabase), mettez une première version en ligne ce soir, puis ajoutez le reste une fois que le produit existe. C'est tout ce dont vous avez besoin. Lancez-vous, et construisez.